Projet demoTP5bis

Explication de chaque fonctionnalité

Application Spring Boot · Spring MVC · Spring Data JPA · Thymeleaf · MySQL

Partie	Contenu
1. Architecture	Comment le projet est organisé, configuration, entités, menu
2. Contacts	Liste, ajout, suppression, modification d'un contact
3. Personnes	Liste, ajout, suppression, modification d'une personne
4. Recherches	Contacts mal notés, recherche par note, propriétaire d'un contact, personnes sans contact, meilleure moyenne
5. Récapitulatif	Tableau de toutes les URL et points à retenir

1. Architecture générale du projet

Le projet est une application web **Spring Boot** organisée selon le modèle **MVC** (Modèle, Vue, Contrôleur). Toute la logique s'exécute **côté serveur, en Java**. Le navigateur ne reçoit que des pages HTML déjà construites : il n'y a aucun JavaScript.

1.1 Les fichiers et leur rôle

Couche	Fichiers	Rôle
Entités (Modèle)	`entities/Personne.java` `entities/Contact.java`	Classes Java qui correspondent aux tables de la base de données
Repositories (DAO)	`dao/PersonneRepository.java` `dao/ContactRepository.java`	Accès à la base : Spring génère automatiquement les requêtes SQL
Contrôleurs	`web/Ctr.java` `web/CtrPersonne.java`	Reçoivent les requêtes HTTP, appellent les repositories et choisissent la page à afficher
Vues (HTML)	`templates/*.html`	Pages Thymeleaf qui affichent les données transmises par le contrôleur
Configuration	`application.properties`	Connexion à MySQL et comportement de Hibernate

1.2 Le cycle d'une requête (valable pour toutes les fonctionnalités)

```
Navigateur --- GET /contacts ---> Contrôleur (méthode annotée @RequestMapping("/contacts"))
                                        |
                                        v
                                  Repository  ---- SQL ---->  Base MySQL
                                        |
                    liste d'objets <----+
                                        |
                                        v
                    model.addAttribute("contacts", ...)
                    return "liste";        --> Thymeleaf remplit templates/liste.html
                                        |
Navigateur <------ page HTML finale ----+
```

- `@Controller` : indique à Spring que la classe contient des méthodes qui traitent des URL.
- `@RequestMapping("/url")` : associe une URL à une méthode. Sans précision, il accepte **GET et POST**.
- `@RequestParam` : récupère un paramètre envoyé par un formulaire ou présent dans l'URL (`?note=15`).
- `Model` : sert de « sac » pour transmettre des données à la page HTML.
- La chaîne renvoyée (`return "liste"`) est le **nom du template** à afficher (`templates/liste.html`).
- `@Autowired` : Spring fournit automatiquement l'objet repository, sans `new`.

1.3 La configuration : application.properties

src/main/resources/application.properties

```
spring.jpa.hibernate.ddl-auto=create
spring.datasource.url=jdbc:mysql://${MYSQL_HOST:localhost}:3306/Contacts
spring.datasource.username=root
spring.datasource.password=
```

- L'application se connecte à la base MySQL **Contacts**, avec l'utilisateur `root` et sans mot de passe.
- `ddl-auto=create` : au démarrage, Hibernate **supprime puis recrée les tables** à partir des entités. Les données sont donc **perdues à chaque redémarrage** de l'application.

1.4 Les entités et leur relation

entities/Personne.java et entities/Contact.java (simplifiés)

```
@Entity
public class Personne {
    @Id @GeneratedValue
    private Long id;                        // clé primaire générée automatiquement
    private String nom, prenom, adresse;

    @OneToMany(mappedBy="personne", cascade=CascadeType.REMOVE)
    private List<Contact> contacts = new ArrayList<Contact>();

    public double getMoyenne() { ... }     // calculée, pas stockée en base
}

@Entity
public class Contact {
    @Id
    private String email;                   // l'email EST la clé primaire
    private int note;

    @ManyToOne
    private Personne personne;              // colonne personne_id dans la table contact
}
```

- Une **personne** peut avoir **plusieurs contacts** (`@OneToMany`). Un **contact** appartient à **une seule personne** (`@ManyToOne`).
- `mappedBy="personne"` : c'est le champ `personne` de `Contact` qui porte la relation. En base, c'est la table `contact` qui contient la clé étrangère `personne_id`.
- `cascade=CascadeType.REMOVE` : quand on supprime une personne, **ses contacts sont supprimés aussi**.
- La clé primaire de `Contact` est l'**email** : deux contacts ne peuvent donc pas avoir le même email. C'est pour ça que `ContactRepository` étend `JpaRepository<Contact, String>`.

1.5 Les repositories : du SQL sans écrire de SQL

```
public interface ContactRepository extends JpaRepository<Contact,String> {
    List<Contact> findByNoteLessThan(int note);
    List<Contact> findByNote(int note);
}
public interface PersonneRepository extends JpaRepository<Personne,Long> {
    List<Personne> findByContactsIsEmpty();
}
```

En héritant de `JpaRepository`, on obtient gratuitement `findAll()`, `findById()`, `save()`, `deleteById()` et `existsById()`. Les autres méthodes sont des **requêtes dérivées** : Spring lit le **nom de la méthode** et construit lui-même la requête SQL.

Méthode	Requête générée (simplifiée)
`findAll()`	`SELECT * FROM contact`
`findById(email)`	`SELECT * FROM contact WHERE email = ?`

Méthode	Requête générée (simplifiée)
`save(c)`	`INSERT` si l'objet est nouveau, sinon `UPDATE`
`deleteById(email)`	`DELETE FROM contact WHERE email = ?`
`findByNote(n)`	`... WHERE note = ?`
`findByNoteLessThan(n)`	`... WHERE note < ?`
`findByContactsIsEmpty()`	personnes qui n'ont aucun contact

1.6 Le menu commun : fragments/navbar.html

templates/fragments/navbar.html

```
<div class="navbar" th:fragment="navbar">
    <a th:href="@{/}">Accueil</a>
    ... menus déroulants Contacts / Personnes / Recherches ...
</div>
```

Le menu est écrit **une seule fois** sous forme de **fragment** Thymeleaf. Chaque page l'inclut avec `<div th:replace="~{fragments/navbar :: navbar}"></div>`. Les menus déroulants sont réalisés **uniquement en CSS** (`static/css/style.css`), sans JavaScript. `th:href="@{/contacts}"` génère l'URL correcte du lien.

1.7 L'accueil : /

web/Ctr.java

```
@RequestMapping("/")
public String index() {return "index";}
```

La méthode n'envoie aucune donnée : elle affiche simplement `index.html`, qui contient le menu et le titre « Page d'accueil des contacts ».

2. Gestion des contacts (web/Ctr.java)

2.1 Afficher la liste des contacts : /contacts

Déclencheur : menu *Contacts* > *Liste des contacts*, qui envoie `GET /contacts`.

web/Ctr.java

```java
@RequestMapping("/contacts")
public String contacts(Model model) {
    List<Contact> contacts = contactRepository.findAll();   // SELECT * FROM contact
    model.addAttribute("contacts", contacts);
    return "liste";
}
```

templates/liste.html

```html
<tr th:each="contact: ${contacts}">
  <td th:text="${contact.email}" />
  <td th:text="${contact.note}" />
  <td th:text="${contact.personne.prenom + ' ' + contact.personne.nom}" />
  <td><a th:href="@{/modifierContact(email=${contact.email})}">Modifier</a></td>
  <td><a th:href="@{/deleteSave(email=${contact.email})}">Supprimer</a></td>
</tr>
```

- `th:each` : crée une ligne de tableau par contact (c'est une boucle).
- `${contact.email}` : Thymeleaf appelle `contact.getEmail()`.
- `${contact.personne.prenom}` : on passe par la relation `@ManyToOne` pour afficher le nom de la personne propriétaire.
- `@{/modifierContact(email=...)}` : construit le lien `/modifierContact?email=xxx`. Chaque ligne a ainsi ses propres liens « Modifier » et « Supprimer ».

Qui fait quoi ? La base renvoie tous les contacts, le contrôleur les transmet à la vue, et Thymeleaf construit le tableau.

2.2 Ajouter un contact : /ajout puis /save

Cette fonctionnalité se fait en **deux temps** : afficher le formulaire, puis enregistrer ce qui a été saisi.

Étape 1 : afficher le formulaire (menu *Contacts* > *Ajouter un contact*)

web/Ctr.java

```java
@RequestMapping("/ajout")
public String ajout(Model model) {
    // il faut la liste des personnes pour la liste déroulante
    model.addAttribute("personnes", personneRepository.findAll());
    return "add";
}
```

templates/add.html

```
<p th:text="${erreur}"></p>
<form th:action="@{/save}" method="post">
  <input type="text" placeholder="Entrez votre email" name="email">
  <input type="text" placeholder="Entrez votre note"  name="note">
  <select name="personneId">
    <option th:each="p : ${personnes}" th:value="${p.id}"
            th:text="${p.prenom + ' ' + p.nom}"></option>
  </select>
  <input type="submit" value="Envoyer">
</form>
```

La liste déroulante **affiche** le prénom et le nom de chaque personne, mais **envoie** son `id` (`th:value`). Le paragraphe `${erreur}` reste vide, sauf en cas d'erreur.

Étape 2 : enregistrer (clic sur « Envoyer », qui envoie `POST /save`)

web/Ctr.java

```
@RequestMapping("/save")
public String save(@RequestParam String email, @RequestParam String note,
                   @RequestParam Long personneId, Model model) {
    if (contactRepository.existsById(email)) {              // email déjà utilisé ?
        model.addAttribute("erreur", "Cet email existe deja !");
        return ajout(model);                                 // on réaffiche le formulaire
    }
    Personne p = personneRepository.findById(personneId).get();
    contactRepository.save(new Contact(email, Integer.parseInt(note), p));
    return contacts(model);                                  // on affiche la liste
}
```

- Les trois `@RequestParam` correspondent aux attributs `name` du formulaire.
- **Vérification :** l'email étant la clé primaire, on vérifie avec `existsById` qu'il n'existe pas déjà. Si c'est le cas, on place un message d'erreur et on rappelle `ajout(model)` pour réafficher le formulaire, ce qui recharge aussi la liste des personnes.
- Sinon, on récupère la `Personne` choisie, on convertit la note texte en entier avec `Integer.parseInt`, puis on enregistre le nouveau `Contact` (`INSERT`).
- `return contacts(model)` : au lieu de dupliquer le code, on réutilise la méthode qui affiche la liste.

> **Attention :** si la note saisie n'est pas un nombre (par exemple « abc »), `Integer.parseInt` lève une exception et l'utilisateur obtient une page d'erreur 500. De plus, l'URL reste `/save` : si on actualise la page (F5), le navigateur renvoie le formulaire.

2.3 Supprimer un contact : /delete puis /deleteSave

On peut supprimer un contact **de deux façons** :

- depuis le menu *Contacts* > *Supprimer un contact* : la page `delete.html` affiche un formulaire où l'on tape l'email ;
- depuis la liste des contacts : le lien « Supprimer » de chaque ligne appelle directement `/deleteSave?email=xxx` (en GET).

web/Ctr.java

```java
@RequestMapping("/delete")
public String delete() { return "delete"; }          // affiche juste le formulaire

@RequestMapping("/deleteSave")
public String deleteSave(@RequestParam String email, Model model) {
    Optional<Contact> contact = contactRepository.findById(email);
    if (contact.isPresent()) {
        contactRepository.deleteById(email);          // DELETE FROM contact WHERE email=?
        model.addAttribute("message", "Le contact " + email + " a ete supprime");
    } else
        model.addAttribute("message", "Aucun contact avec cet email");
    return "delete";
}
```

templates/delete.html

```html
<p th:text="${message}"></p>
<form th:action="@{/deleteSave}" method="post">
  <input type="text" placeholder="Email" name="email">
  <input type="submit" value="Envoyer">
</form>
```

- `findById` renvoie un `Optional<Contact>`, c'est-à-dire une « boîte » qui contient peut-être un contact. `isPresent()` indique si le contact existe.
- Dans les deux cas, on revient sur `delete.html` avec un message de confirmation ou d'erreur.
- Le même `/deleteSave` fonctionne en **POST** (formulaire) comme en **GET** (lien), car `@RequestMapping` accepte les deux.

2.4 Modifier un contact : /modifierContact puis /updateContact

Étape 1 : le lien « Modifier » de la liste appelle `/modifierContact?email=xxx`.

web/Ctr.java

```java
@RequestMapping("/modifierContact")
public String modifierContact(@RequestParam String email, Model model) {
    Contact c = contactRepository.findById(email).get();
    model.addAttribute("contact", c);                              // valeurs actuelles
    model.addAttribute("personnes", personneRepository.findAll());  // pour la liste déroulante
    return "modifContact";
}
```

templates/modifContact.html

```html
<form th:action="@{/updateContact}" method="post">
  <input type="text" name="email" th:value="${contact.email}" readonly>
  <input type="text" name="note"  th:value="${contact.note}">
  <select name="personneId">
    <option th:each="p : ${personnes}" th:value="${p.id}" th:text="${p.prenom + ' ' + p.nom}"
            th:selected="${p.id == contact.personne.id}"></option>
  </select>
  <input type="submit" value="Envoyer">
</form>
```

- `th:value` **pré-remplit** les champs avec les valeurs actuelles du contact.
- `readonly` : l'email ne peut pas être modifié, puisque c'est la clé primaire. Il est quand même envoyé avec le formulaire, ce qui permet de savoir quel contact modifier.
- `th:selected` : dans la liste déroulante, la personne actuelle du contact est déjà sélectionnée.

Étape 2 : « Envoyer » appelle `POST /updateContact`.

web/Ctr.java

```java
@RequestMapping("/updateContact")
public String updateContact(@RequestParam String email, @RequestParam String note,
                            @RequestParam Long personneId, Model model) {
    Contact c = contactRepository.findById(email).get();            // on recharge le contact
    c.setNote(Integer.parseInt(note));                              // on change ses valeurs
    c.setPersonne(personneRepository.findById(personneId).get());
    contactRepository.save(c);                                      // UPDATE
    return contacts(model);
}
```

`save()` fait un `UPDATE` et non un `INSERT`, parce que le contact existe déjà en base (même clé primaire). C'est la même méthode `save` que pour l'ajout : JPA choisit l'opération tout seul.

3. Gestion des personnes (web/CtrPersonne.java)

Ce contrôleur fonctionne exactement comme celui des contacts, mais pour l'entité `Personne`. La différence principale est que la clé primaire est un `Long id` généré automatiquement.

3.1 Afficher la liste des personnes : /personnes

web/CtrPersonne.java

```
@RequestMapping("/personnes")
public String personnes(Model model) {
    List<Personne> personnes = personneRepository.findAll();
    model.addAttribute("personnes", personnes);
    return "personnes";
}
```

templates/personnes.html

```
<tr th:each="p: ${personnes}">
  <td th:text="${p.id}" />  <td th:text="${p.nom}" />  ...
  <td th:text="${p.contacts.size()}" />
  <td><a th:href="@{/modifierPersonne(id=${p.id})}">Modifier</a></td>
  <td><a th:href="@{/deletePersonne(id=${p.id})}">Supprimer</a></td>
</tr>
```

La colonne « Nb contacts » utilise `${p.contacts.size()}` : Thymeleaf accède à la liste des contacts de la personne grâce à la relation `@OneToMany`, et JPA va chercher ces contacts en base au moment de l'affichage.

3.2 Ajouter une personne : /ajoutPersonne puis /savePersonne

web/CtrPersonne.java

```
@RequestMapping("/ajoutPersonne")
public String ajoutPersonne() { return "addPersonne"; }

@RequestMapping("/savePersonne")
public String savePersonne(@RequestParam String nom, @RequestParam String prenom,
                           @RequestParam String adresse, Model model) {
    personneRepository.save(new Personne(nom, prenom, adresse));
    return personnes(model);
}
```

Le formulaire `addPersonne.html` contient trois champs (`nom`, `prenom`, `adresse`) envoyés en POST vers `/savePersonne`. Il n'y a pas de vérification de doublon, car l'`id` est généré par la base (`@GeneratedValue`) : chaque nouvelle personne reçoit un identifiant unique.

3.3 Supprimer une personne : /deletePersonne

web/CtrPersonne.java

```
@RequestMapping("/deletePersonne")
public String deletePersonne(@RequestParam Long id, Model model) {
    if (personneRepository.existsById(id))
        personneRepository.deleteById(id);
    return personnes(model);
}
```

- Cette méthode est appelée par le lien « Supprimer » de la liste (`/deletePersonne?id=3`).
- `existsById` évite une erreur si l'id n'existe pas.

- Grâce à `cascade=CascadeType.REMOVE` dans `Personne`, **tous les contacts de la personne sont supprimés avec elle**. Sans cette cascade, la base refuserait la suppression, car des contacts feraient encore référence à cette personne (clé étrangère).

3.4 Modifier une personne : /modifierPersonne puis /updatePersonne

web/CtrPersonne.java

```
@RequestMapping("/modifierPersonne")
public String modifierPersonne(@RequestParam Long id, Model model) {
    model.addAttribute("personne", personneRepository.findById(id).get());
    return "modifPersonne";
}
@RequestMapping("/updatePersonne")
public String updatePersonne(@RequestParam Long id, @RequestParam String nom,
                             @RequestParam String prenom, @RequestParam String adresse,
                             Model model) {
    Personne p = personneRepository.findById(id).get();
    p.setNom(nom); p.setPrenom(prenom); p.setAdresse(adresse);
    personneRepository.save(p);                                    // UPDATE
    return personnes(model);
}
```

templates/modifPersonne.html

```
<input type="hidden" name="id" th:value="${personne.id}">
<input type="text" name="nom" th:value="${personne.nom}"> ...
```

Le principe est le même que pour la modification d'un contact. Ici, l'`id` est transmis dans un champ **caché** (`type="hidden"`) : l'utilisateur ne le voit pas, mais il est envoyé avec le formulaire, ce qui permet au contrôleur de savoir quelle personne mettre à jour.

4. Les recherches

Toutes les recherches sont accessibles depuis le menu *Recherches*. Elles se font toutes **en Java, côté serveur**. Selon le cas, le filtrage est fait soit **par la base de données** (requête dérivée), soit **par une boucle Java**.

4.1 Contacts mal notés : /malNotes

web/Ctr.java

```java
@RequestMapping("/malNotes")
public String malNotes(Model model) {
    List<Contact> contacts = contactRepository.findByNoteLessThan(10);
    model.addAttribute("contacts", contacts);
    return "malnotes";
}
```

- Il n'y a pas de formulaire : le seuil **10** est écrit directement dans le code.
- `findByNoteLessThan(10)` : Spring lit le nom de la méthode, `findBy` + `Note` + `LessThan`, et génère `SELECT * FROM contact WHERE note < 10`.
- `malnotes.html` affiche les contacts trouvés avec `th:each`, comme la liste des contacts.

Qui fait quoi ? Le filtrage est fait par la **base de données**.

4.2 Rechercher les contacts par note : /recherche

1. Le formulaire (`recherche.html`)

templates/recherche.html

```html
<form th:action="@{/recherche}" method="get">
  <input type="text" placeholder="Entrez une note" name="note">
  <input type="submit" value="Rechercher">
</form>
```

En cliquant sur « Rechercher », le navigateur envoie `GET /recherche?note=15`. Comme le formulaire est en **GET**, la note apparaît dans l'URL.

2. Le contrôleur

web/Ctr.java

```java
@RequestMapping("/recherche")
public String recherche(@RequestParam(required = false) Integer note, Model model) {
    if (note != null) {
        List<Contact> contacts = contactRepository.findByNote(note);
        model.addAttribute("contacts", contacts);
    }
    return "recherche";
}
```

- `required = false` : on peut ouvrir la page sans paramètre (depuis le menu). `note` vaut alors `null`, aucune recherche n'est faite et seul le formulaire s'affiche.
- Le type `Integer` (et pas `int`) est obligatoire pour pouvoir valoir `null`.
- **La même méthode et la même page** servent à afficher le formulaire et le résultat.

3. Le repository : `findByNote(int note)` génère `SELECT * FROM contact WHERE note = ?`.

4. L'affichage

templates/recherche.html

```
<div th:if="${contacts != null}">
  <table> ... <tr th:each="contact: ${contacts}"> ... </tr> </table>
</div>
```

`th:if` : le tableau n'est affiché que si une recherche a été faite, c'est-à-dire si le contrôleur a ajouté `contacts` au modèle.

Qui fait quoi ? Le HTML sert à **saisir** la note et à **afficher** le résultat. Le **contrôleur Java** reçoit la note et la **base de données** fait le filtrage.

Remarque : le champ est en `type="text"`. Si on tape « abc », Spring ne peut pas convertir la valeur en `Integer` et renvoie une erreur 400. Avec `type="number"`, le navigateur bloquerait cette saisie avant l'envoi.

4.3 Trouver le propriétaire d'un contact : /proprietaire

web/Ctr.java

```
@RequestMapping("/proprietaire")
public String proprietaire(@RequestParam(required = false) String email, Model model) {
    if (email != null) {
        Optional<Contact> contact = contactRepository.findById(email);
        if (contact.isPresent())
            model.addAttribute("personne", contact.get().getPersonne());
        else
            model.addAttribute("erreur", "Aucun contact avec cet email");
    }
    return "proprietaire";
}
```

- Le fonctionnement est le même que pour la recherche par note : un formulaire en GET (champ `email`), un paramètre facultatif, et une seule page pour le formulaire et le résultat.
- On cherche le contact par sa clé primaire (`findById`), puis on **remonte la relation** `@ManyToOne` avec `getPersonne()` pour obtenir son propriétaire.
- Dans `proprietaire.html`, `<p th:text="${erreur}">` affiche l'erreur éventuelle, et `th:if="${personne != null}"` n'affiche le tableau que si une personne a été trouvée.

4.4 Personnes sans contact : /sansContact

web/CtrPersonne.java

```
@RequestMapping("/sansContact")
public String sansContact(Model model) {
    model.addAttribute("personnes", personneRepository.findByContactsIsEmpty());
    return "sanscontact";
}
```

- `findByContactsIsEmpty()` est une requête dérivée : `Contacts` désigne la liste `contacts` de `Personne`, et `IsEmpty` signifie « vide ».
- Spring génère une requête qui ne garde que les personnes **sans aucun contact associé** (en SQL : `WHERE NOT EXISTS (SELECT ... FROM contact WHERE personne_id = p.id)`).
- `sanscontact.html` affiche ces personnes avec `th:each`.

Qui fait quoi ? Le filtrage est fait par la **base de données**.

4.5 Personne avec la meilleure moyenne : /meilleureMoyenne

Cette fois, **le calcul ne se fait pas dans la base de données mais en Java**. On récupère toutes les personnes, puis le code Java calcule les moyennes et les compare dans une boucle.

1. Le lien : il n'y a rien à saisir, donc pas de formulaire. Le menu envoie simplement `GET /meilleureMoyenne`.

2. Le contrôleur

web/CtrPersonne.java

```
@RequestMapping("/meilleureMoyenne")
public String meilleureMoyenne(Model model) {
    List<Personne> personnes = personneRepository.findAll();   // SELECT * FROM personne
    Personne meilleure = null;
    for (Personne p : personnes) {
        if (p.getContacts().size() > 0) {        // on ignore ceux sans contact
            if (meilleure == null || p.getMoyenne() > meilleure.getMoyenne())
                meilleure = p;                   // nouveau meilleur
        }
    }
    model.addAttribute("personne", meilleure);
    return "meilleure";
}
```

C'est l'algorithme classique de **recherche du maximum** :

- `meilleure` part de `null` : on n'a encore trouvé personne.
- Pour chaque personne qui a au moins un contact, on la garde si c'est la première ou si sa moyenne dépasse celle du meilleur actuel.
- À la fin de la boucle, `meilleure` contient la personne qui a la plus grande moyenne.

3. Le calcul de la moyenne, dans l'entité `Personne` :

entities/Personne.java

```
public double getMoyenne() {
    if (contacts.size() == 0) return 0;
    int total = 0;
    for (Contact c : contacts) total = total + c.getNote();
    return (double) total / contacts.size();
}
```

- La moyenne **n'est pas enregistrée en base**, puisqu'il n'y a pas de colonne `moyenne`. Elle est recalculée à chaque appel à partir de la liste `contacts`.
- Cette liste vient de la relation `@OneToMany`. JPA va chercher les contacts de la personne en base au moment où on les utilise pour la première fois.
- Le `(double)` est indispensable : sans lui, `total / size` serait une division entière (15 / 2 donnerait 7 au lieu de 7.5).

4. L'affichage (`meilleure.html`)

templates/meilleure.html

```
<div th:if="${personne != null}">
  ...
  <td th:text="${personne.moyenne}" />
</div>
```

`${personne.moyenne}` fonctionne alors qu'il n'existe aucun attribut `moyenne`. Thymeleaf appelle en fait la méthode `getMoyenne()` : il traduit `xxx` en `getXxx()`. Si aucune personne n'a de contact, `meilleure` reste `null` et le tableau n'est pas affiché.

Qui fait quoi ? Le calcul est fait par **Java** (boucle dans le contrôleur et `getMoyenne()`). La base se contente de renvoyer toutes les personnes et tous leurs contacts.

À retenir : en cas d'égalité, c'est la **première** personne de la liste qui est gardée, car la comparaison utilise `>` et non `>=`. Comme tout est chargé en mémoire, cette méthode convient à un TP mais deviendrait lente avec beaucoup de données. On pourrait plutôt faire calculer la moyenne par la base avec une requête `@Query` : `SELECT c.personne FROM Contact c GROUP BY c.personne ORDER BY AVG(c.note) DESC LIMIT 1`.

4.6 Comparaison des recherches

Recherche	Saisie	Qui fait le travail ?	Méthode utilisée
Contacts mal notés	non	Base de données	`findByNoteLessThan(10)`
Recherche par note	oui (note)	Base de données	`findByNote(note)`
Propriétaire d'un contact	oui (email)	Base, puis relation JPA	`findById(email)` puis `getPersonne()`
Personnes sans contact	non	Base de données	`findByContactsIsEmpty()`
Meilleure moyenne	non	**Java** (boucle)	`findAll()` puis `getMoyenne()`

5. Récapitulatif

5.1 Toutes les URL de l'application

URL	Contrôleur	Paramètres	Page affichée	Action
`/`	Ctr	-	`index`	Accueil
`/contacts`	Ctr	-	`liste`	Liste des contacts
`/ajout`	Ctr	-	`add`	Formulaire d'ajout de contact
`/save`	Ctr	email, note, personneId	`liste` ou `add`	Enregistre un contact
`/delete`	Ctr	-	`delete`	Formulaire de suppression
`/deleteSave`	Ctr	email	`delete`	Supprime un contact
`/modifierContact`	Ctr	email	`modifContact`	Formulaire de modification
`/updateContact`	Ctr	email, note, personneId	`liste`	Met à jour un contact
`/malNotes`	Ctr	-	`malnotes`	Contacts avec note < 10
`/recherche`	Ctr	note (facultatif)	`recherche`	Contacts ayant cette note
`/proprietaire`	Ctr	email (facultatif)	`proprietaire`	Personne qui possède le contact
`/personnes`	CtrPersonne	-	`personnes`	Liste des personnes
`/ajoutPersonne`	CtrPersonne	-	`addPersonne`	Formulaire d'ajout de personne
`/savePersonne`	CtrPersonne	nom, prenom, adresse	`personnes`	Enregistre une personne
`/deletePersonne`	CtrPersonne	id	`personnes`	Supprime une personne et ses contacts
`/modifierPersonne`	CtrPersonne	id	`modifPersonne`	Formulaire de modification
`/updatePersonne`	CtrPersonne	id, nom, prenom, adresse	`personnes`	Met à jour une personne
`/sansContact`	CtrPersonne	-	`sanscontact`	Personnes sans contact
`/meilleureMoyenne`	CtrPersonne	-	`meilleure`	Personne avec la meilleure moyenne

5.2 Les attributs Thymeleaf utilisés

Attribut	Rôle	Exemple dans le projet
`th:text`	Affiche une valeur	`th:text="${contact.email}"`

Attribut	Rôle	Exemple dans le projet
`th:each`	Boucle : une répétition par élément	`th:each="contact: ${contacts}"`
`th:if`	N'affiche l'élément que si la condition est vraie	`th:if="${personne != null}"`
`th:href`	Construit l'URL d'un lien	`@{/modifierPersonne(id=${p.id})}`
`th:action`	Construit l'URL d'envoi d'un formulaire	`th:action="@{/save}"`
`th:value`	Pré-remplit un champ ou fixe la valeur d'une option	`th:value="${contact.note}"`
`th:selected`	Sélectionne une option de liste déroulante	`${p.id == contact.personne.id}`
`th:replace`	Insère un fragment (le menu)	`~{fragments/navbar :: navbar}`

5.3 Points à retenir

- Toute la logique est en **Java, côté serveur**. Le HTML (Thymeleaf) sert seulement à **saisir** et à **afficher**.
- Le parcours est toujours le même : **formulaire ou lien**, puis **contrôleur**, puis **repository**, puis **base**, puis retour au **modèle** et enfin à la **page HTML**.
- Les méthodes `findBy...` des repositories sont **générées par Spring à partir de leur nom**.
- Seule la **meilleure moyenne** est calculée en Java. Les autres filtrages sont faits par la base.
- `save()` sert à la fois à **ajouter** (`INSERT`) et à **modifier** (`UPDATE`).
- La suppression d'une personne entraîne celle de ses contacts (`CascadeType.REMOVE`).
- Avec `ddl-auto=create`, la base est **vidée à chaque redémarrage**.